

AWS Deployment Service Integration

Architecture, Security Model, and Implementation Plan for Automated Infrastructure Orchestration.



Operational Objective: On-Demand Infrastructure Automation



Core Goal

Automate the setup and teardown of AWS environments via a new RESTful 'HTML Service'.



The Strategy

Separation of Concerns: Decouple the orchestration logic (The 'Brain') from the low-level execution (The 'Hands').



Security

Zero-Trust Security: No plaintext secrets stored or transmitted; credentials are **encrypted per-transaction.**

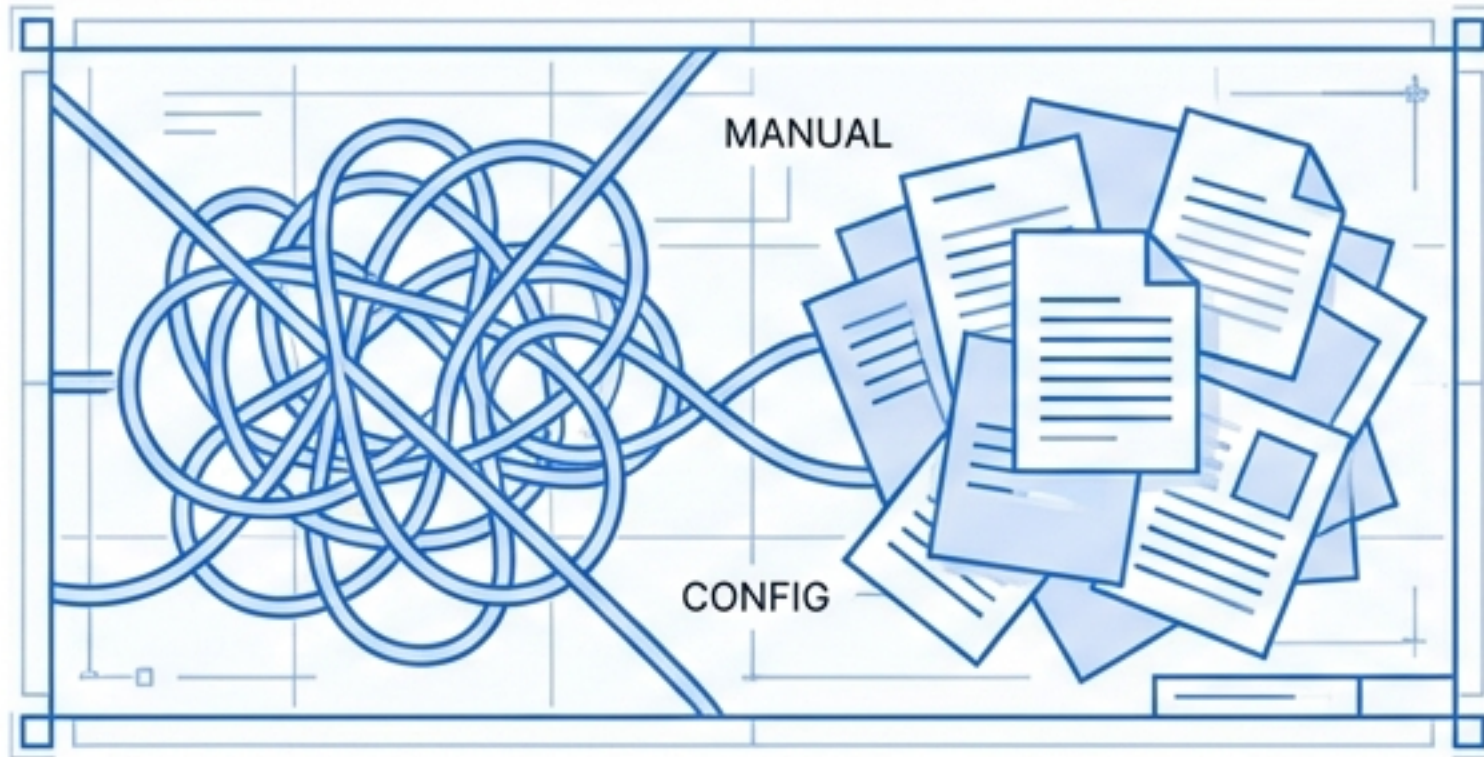


Technology

Built upon the existing OSBot-AWS library to leverage simplified, type-safe Python wrappers.

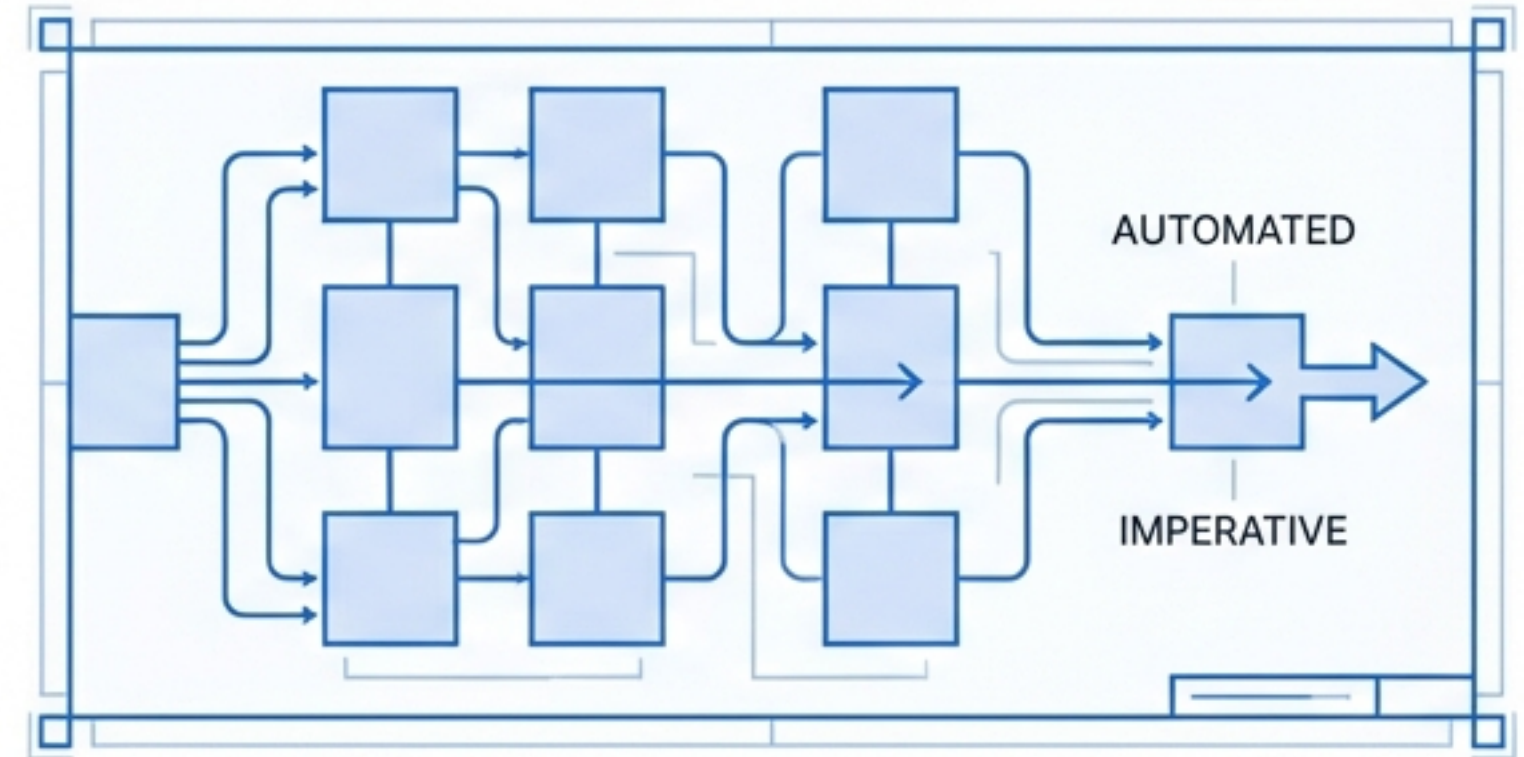
The Shift: From Manual Configuration to Imperative Control

Current State (The Bottleneck)



- Reliance on manual setup and CLI commands
- Risk of configuration drift
- Potential for human error
- Orphaned resources increasing costs

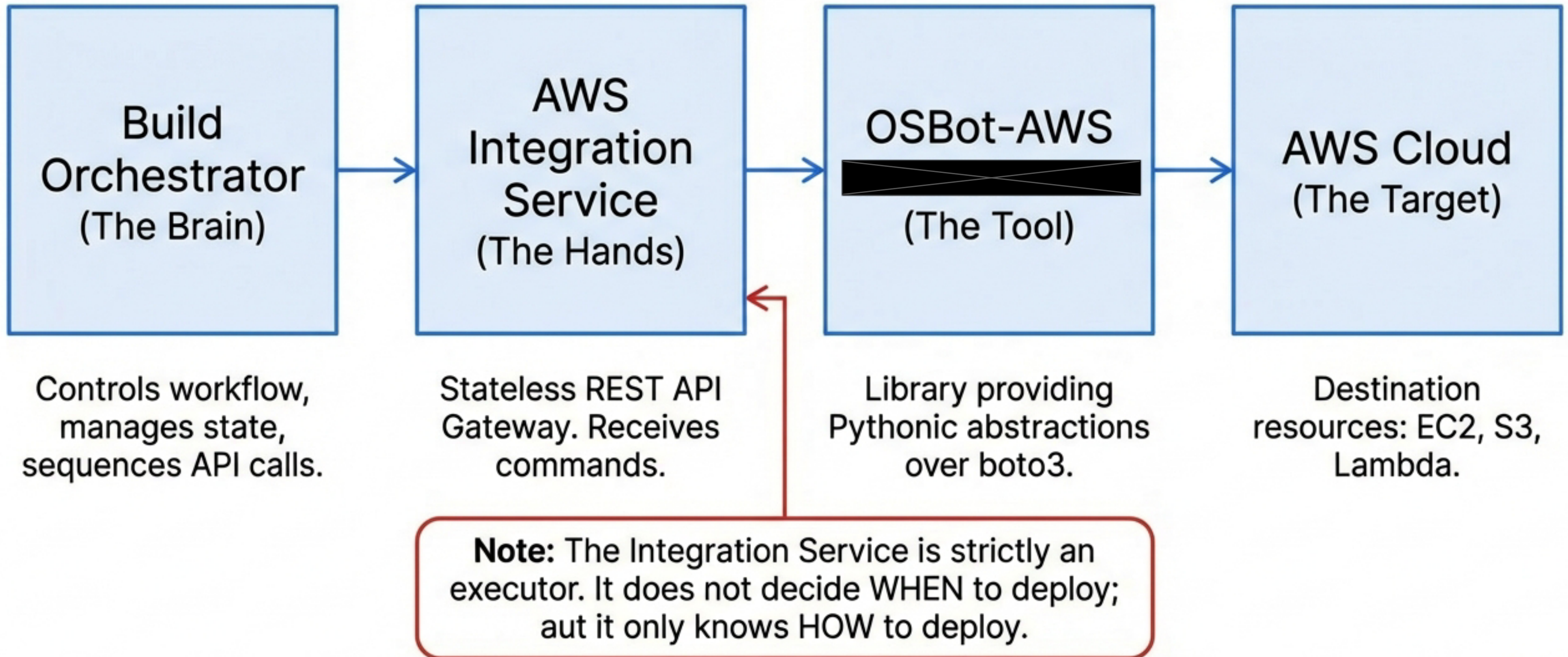
Future State (The Solution)



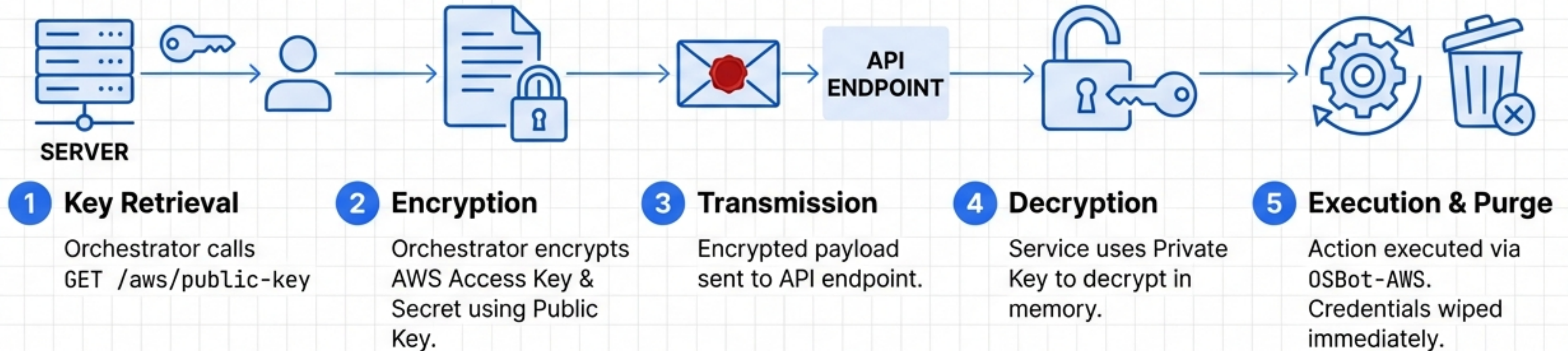
- One-click provisioning
- Immutable infrastructure (baking AMIs)
- Automated teardown (hygiene)
- Deployment logic defined in code

Insight: We are building a custom deployment engine analogous to [CloudFormation](#), but with [imperative control](#) tailored to our **security requirements**.

System Architecture & Roles



Zero-Trust Security Model: The Credential Workflow



Security Note: Credentials ideally use Least Privilege IAM roles, limiting the blast radius.

API Design Philosophy: Fine-Grained Primitives

Concept: We expose atomic actions, not monolithic workflows. This allows the Orchestrator to compose custom sequences for different environments (Dev vs. Prod).

“The AWS Service provides the instruments; the Build Orchestrator is the conductor.”

Storage

S3 Bucket
management

Compute

EC2 instances,
AMI baking

Scaling

Load Balancers,
Auto Scaling
Groups

Serverless

Lambda functions,
URL
configurations

Core Primitives: Storage & Compute

S3 Endpoints

POST /aws/s3/bucket/create

Idempotent creation (handles existing buckets).

GET /aws/s3/bucket/check

Verifies existence/properties.

DELETE /aws/s3/bucket/delete

Ensures empty bucket (delete objects/versions first) then removes bucket.

EC2 & AMI Endpoints

POST /aws/ec2/ami/create

Snapshots an instance to bake a new AMI.

POST /aws/ec2/instance/launch

Boots an instance (e.g., for testing or baking).

POST /aws/ec2/instance/run-command

Executes configuration scripts via AWS SSM.

Core Primitives: Scaling & Serverless

Scaling Endpoints

POST /aws/target-group/create

Prepares network routing for Load Balancer.

POST /aws/autoscaling/create

Complex primitive: Links AMI + Target Group + Launch Template.

GET /aws/autoscaling/status

Verification for health checks.

Lambda Endpoints

POST /aws/lambda/create

Uploads code/config. Supports 'enable_url: true'.

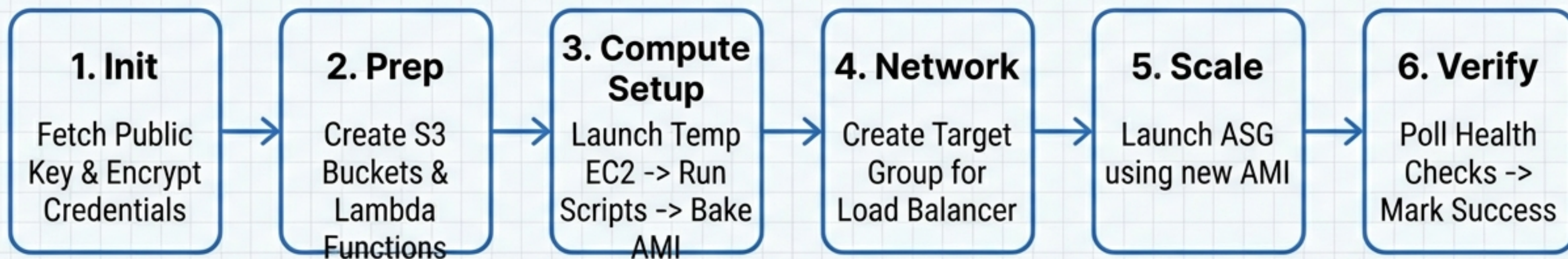
GET /aws/lambda/check

Verifies function readiness.

DELETE /aws/lambda/delete

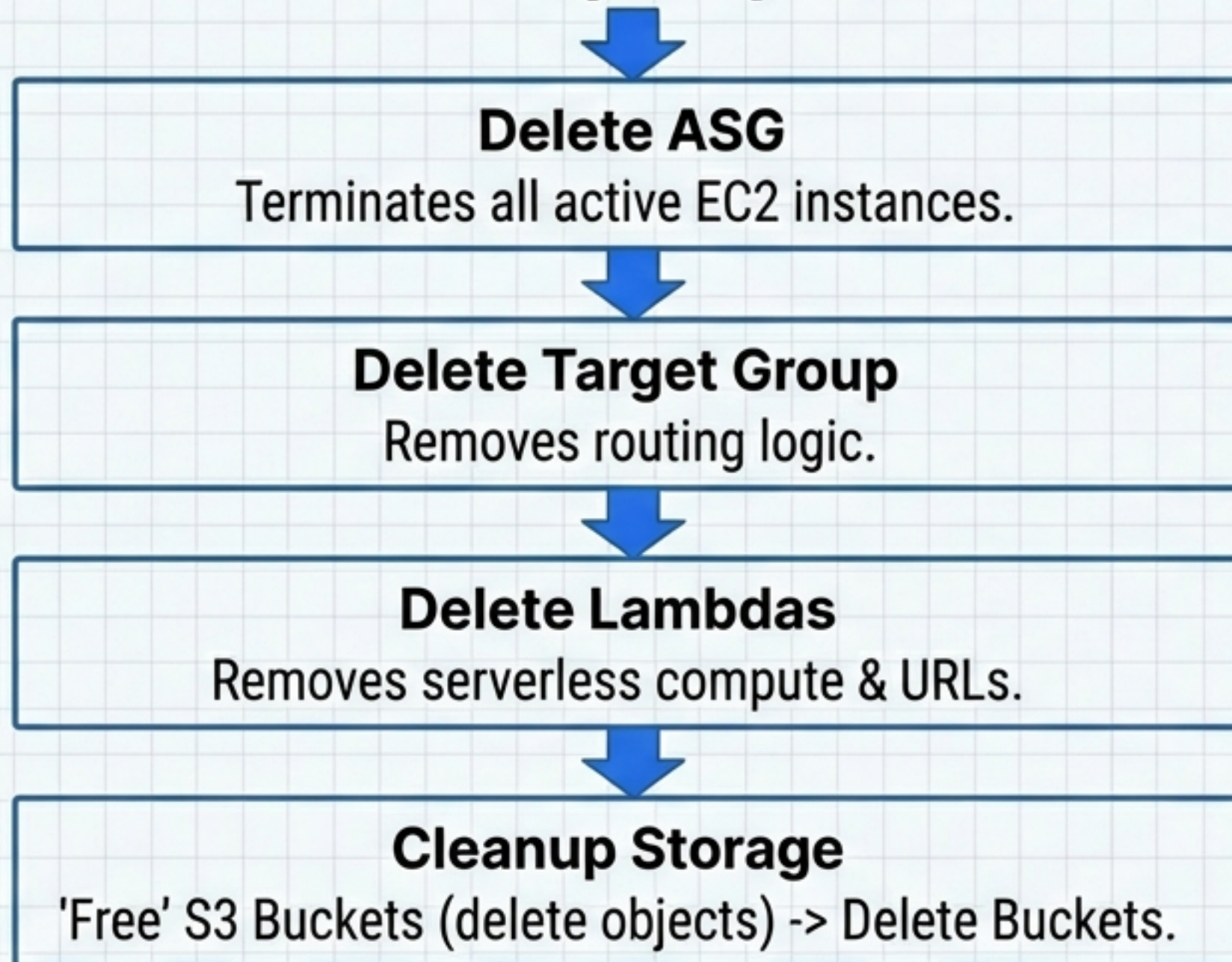
Removes function and associated URL bindings.

Orchestration in Action: The Deployment Workflow



Teardown & Hygiene: The Reverse Flow

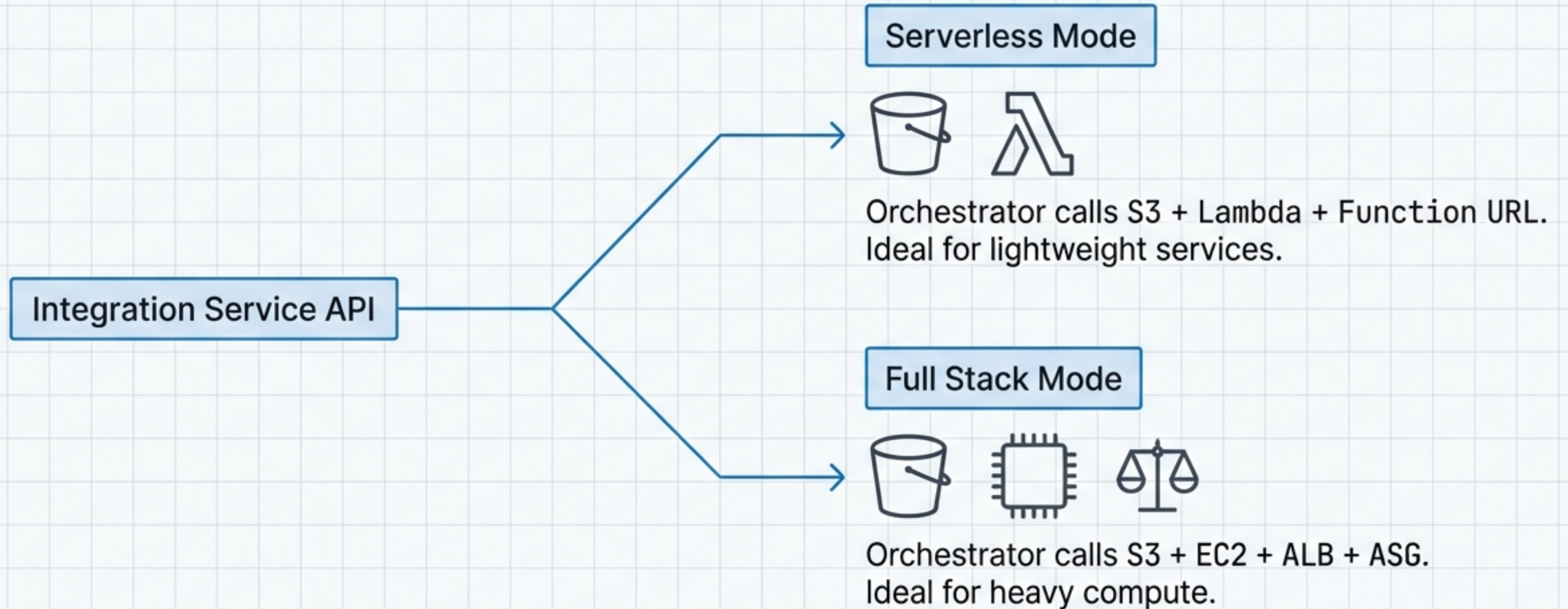
Cleanup Sequence



Reliability:

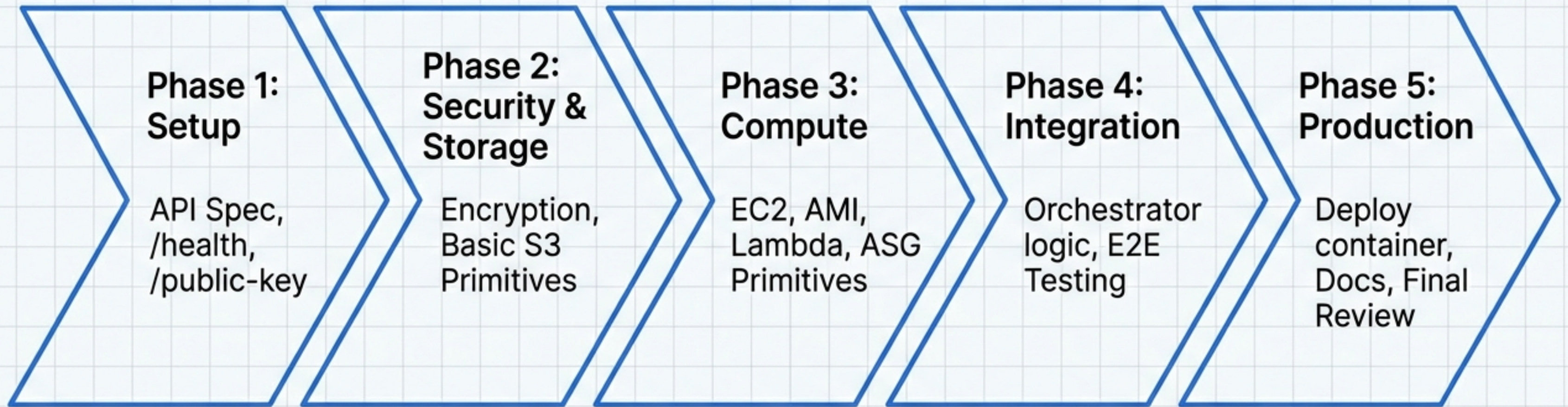
Orchestrator automatically triggers this flow upon partial failure to prevent 'zombie' infrastructure.

Multi-Mode Deployment Flexibility



Key Takeaway: The AWS Service does not enforce the topology. It simply executes the primitives requested by the Orchestrator.

Implementation Roadmap



Technical Requirements & Stack



Language

Python (leveraging osbot-aws, potentially osbot-fast-api).



Hosting

Docker container or Serverless function behind **API Gateway**.



Communication

HTTPS (TLS) enforced for all transport.



Authentication

- Internal **API Tokens** (Header-based) for **Orchestrator** auth.
- **Payload Encryption** (RSA) for AWS Credentials.



Logging

Operational logging (Success/Fail)- **NO** decrypted secrets exposed.

Conclusion & Immediate Next Steps

Summary

We are building a secure, internal infrastructure engine. It empowers us to spin up ephemeral test environments and ensure production consistency via code.

Immediate Action

Finalise the API Specification document and initialize the code repository.

Closing Statement

Automating the path to production with zero-trust security.

Appendix: API Payload Structure

```
POST /aws/ec2/instance/launch
```

```
{
```

```
  "ami_id": "ami-05678",
```

```
  "instance_type": "t3.medium",
```

```
  "security_group": "sg-12345",
```

```
  "credentials": "<encrypted_blob_EyJhbGciOiJ...>"
```

```
}
```

Encrypted blob
ensures stateless
authentication.